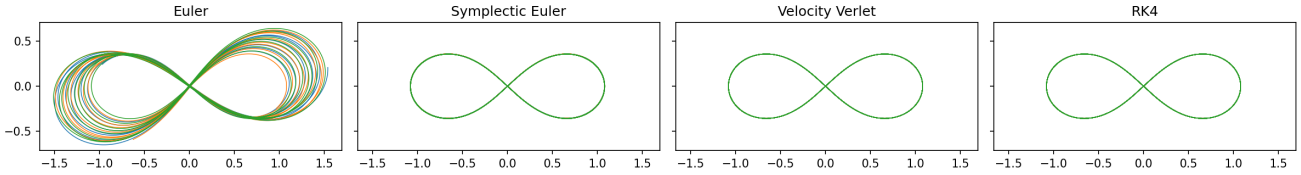
PROJECT MANUAL

Orbit Lab

A gravity simulator that tests how trustworthy numerical methods really are

Figure-8 after 10 periods



Same three-star figure-8 orbit, four different methods, 10 laps. Only the careful methods keep the shape.

4	3	9	50
numerical methods compared	experiments, from 2 bodies up to 9	bodies in the Solar System run	years of planetary motion simulated

OVERVIEW

What is Orbit Lab?

Orbit Lab is a gravity simulator written in Python. It predicts how planets and stars move when they pull on each other, and then does something more important: it **races four different ways of doing the math** and checks each answer against the laws of physics. Think of it as a fair test of four calculators, graded by one rule that nature never breaks: **total energy stays constant**.

INSPIRATION

Why we built it

This project combines two courses. **Computer Simulation** taught how to model a physical system step by step through time, starting with a ballistic projectile simulator. **Numerical Methods** supplied the tools to measure accuracy: integrator design, order of convergence and error analysis. Orbit Lab is where the two meet: simulate something real, then rigorously test how far the simulation can be trusted.

THE CORE IDEA

How a simulation works

STEP 1	STEP 2	STEP 3	STEP 4
Look	**Compute**	**Step**	**Repeat**
Read where every object is and how fast it moves.	Use Newton's law to find the gravity pulling on each one.	Move everything forward a tiny slice of time (dt).	Do it thousands of times, like pages of a flipbook.

THE SCIENCE

Gravity, and why we need a computer

Every object with mass pulls on every other object. Newton wrote the pull between two objects as:

$$F = G \times m_1 \times m_2 / r^2$$

Here m_1 and m_2 are the two masses, **r** is the distance between them and **G** is a constant that sets gravity's strength. For two objects, mathematicians found exact orbits long ago. For **three or more, no general formula exists**, and tiny changes can grow into huge differences (chaos). So scientists let computers find the motion one small step at a time.

THE CONTENDERS

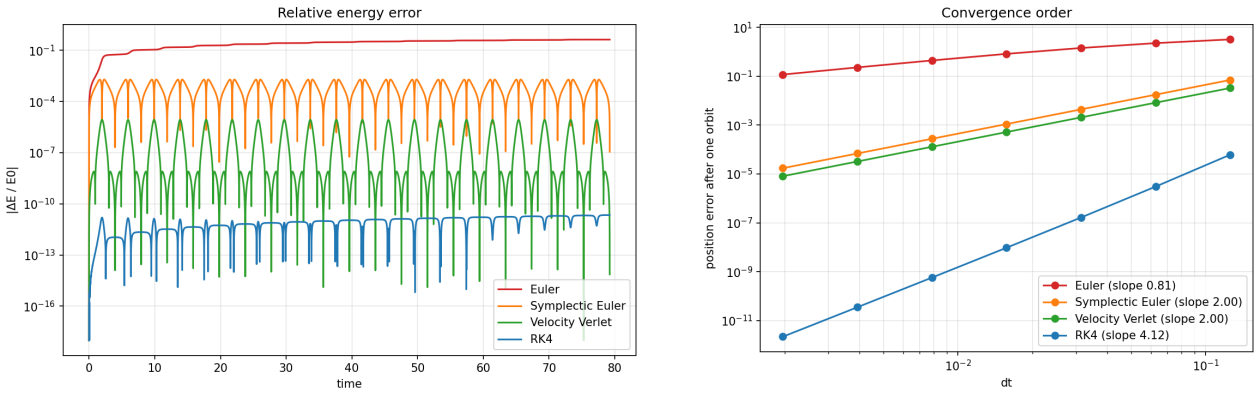
Four methods, one race

Method	The idea in one sentence	What happens over time
Euler	Move with the current speed, then update the speed.	Gains a little energy every step, so orbits **spiral outward**.
Symplectic Euler	Same, but update the speed first.	One small swap makes it far more **stable**.
Velocity Verlet	Average the pull at the start and end of each step.	Very accurate; error stays **small forever**.
RK4 (Runge-Kutta)	Sample the pull at four points inside each step and blend.	Superb short-term accuracy; error **slowly builds up**.

THE REPORT CARD

Energy tells us who is honest

In a real isolated system, total energy never changes. So if a simulation's energy **drifts**, the simulation is making mistakes. We plot this drift on a log scale, where each gridline is ten times bigger than the last. **Lower is better.** A second test halves the time step and checks that each method improves at its expected rate (its *order*).



Left: energy error over time. Euler climbs steadily; Verlet stays bounded. Right: error vs. time step; the slopes match theory.

RESULTS

What we found

Final relative energy error. **Smaller is better.** Red marks the failure, green marks the winners.

Method	2-body orbit (20 orbits)	Figure-8 (10 periods)	Order measured
Euler	4.3×10^{-1}	3.0×10^{-1}	0.8
Symplectic Euler	1.1×10^{-7}	3.5×10^{-5}	2.0*
Velocity Verlet	7×10^{-15}	4.6×10^{-9}	2.0
RK4	2.2×10^{-11}	8.4×10^{-12}	4.1

* Symplectic Euler measured second order on the circular orbit because its leading error cancels after exactly one lap on that symmetric path. We report the measurement as it came out.

- **Plain Euler is unfit for orbits.** It steadily gains energy, so planets drift away from the truth.
- **RK4 wins the sprint, Verlet wins the marathon.** Over 50 Solar System years RK4's error (2.8×10^{-8}) ended larger than Verlet's (3.4×10^{-9}), because RK4's error keeps growing while Verlet's stays bounded.
- **The figure-8 is delicate, not fragile.** Nudge one star by 0.001 and the shape survives, but the three stars slowly drift out of step (the gap grew from 0.08 to about 0.8 over 40 laps).

WHY IT MATTERS

The same ideas power real science

- **Spacecraft navigation:** paths are predicted by numerically integrating the forces on the craft.
- **Solar System stability:** long runs show chaos; energy-conserving (symplectic) methods are a key tool.
- **Galaxies and cosmology:** simulations follow huge numbers of particles using fast tree algorithms.
- **Beyond space:** weather, engineering, finance and games share one lesson: the algorithm you choose decides whether you can trust the answer.

GET STARTED

Run it in three commands

```
pip install -r requirements.txt
python two_body.py
python run_examples.py
```

Keep `nbody.py` in the same folder (it is a library, never run it directly). Close the plot windows to finish. `run_examples.py` may take a minute or two.

```
Orbit_Lab/
  nbody.py          physics + 4 methods
  two_body.py       Part 1: one orbit
  run_examples.py   Parts 2 and 3
  requirements.txt  libraries
  images/           results
```

HONEST LIMITS AND NEXT STEPS

Limits: 2D only, point masses, Newtonian gravity, and a simplified Solar System (circular orbits at random angles, not real-date positions). Each step checks every pair, so it suits tens of bodies, not thousands.

Next: automated tests, a 3D version, adaptive time steps, the Barnes-Hut algorithm for a galaxy-collision demo, and a 5,000-year Solar System run to watch RK4's drift overtake Verlet's.